An Automated Framework for Dataset Quality Assessment and Data Leakage Detection in Machine Learning

Master's Thesis Defense

Jaime Cano Moraño

ETSIT — Universidad Politécnica de Madrid
Illinois Institute of Technology

July 2026

The Problem: Models Fail Because of Data, Not Algorithms

- Most ML effort targets **models**; most production failures trace back to **data**: missing values, duplicates, imbalance, drift … and, most insidiously, **data leakage**.
- **Data leakage**: information about the target reaches the training features in a way that will not exist at prediction time.
- Leakage is dangerous because it is *silent*: the model looks **better**, not worse — inflated validation metrics that collapse in production.
- Documented across medicine, finance, and academic reproducibility studies (Kaufman et al. 2012; Kapoor & Narayanan).

Core question of this thesis

Can dataset quality assessment and leakage detection be **automated, quantified, and made actionable** in a single tool?

A Motivating Example

A logistic regression classifier trained on a dataset with an engineered leaky feature:

With leakage (as delivered)

Validation accuracy: **1.000**

After removing leaky features

Validation accuracy: **0.952**

- That $\Delta = -0.048$ is the gap between a reproducible result and a broken one.
- A human reviewer might catch this. **At scale, nobody reviews every feature of every dataset.**
- The framework presented today detects this automatically — and quantifies the damage.

The Gap in Existing Tools

- Mature open-source tools exist for data *profiling* and *validation*: **ydata-profiling**, **Great Expectations**, **Deepchecks**.
- They cover descriptive statistics and rule-based validation well ...
- ... but **leakage detection is almost absent**: on four constructed leakage scenarios, they detect **at most 1 of 4** (and only via manually authored rules).
- None produces a single interpretable *readiness* verdict or code-level fixes.

Positioning

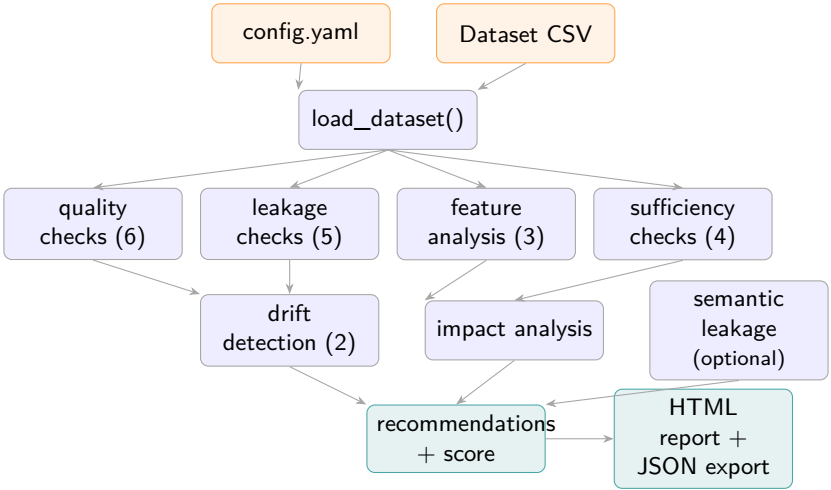
The framework **complements** these tools — it fills the leakage-detection and actionability gap rather than replacing general-purpose profiling.

Objectives and Contributions

1. **A unified, automated framework**: 20 deterministic checks across 5 dimensions (quality, leakage, features, sufficiency, drift) + impact analysis, with CLI, Python SDK, and web interfaces.
2. **A unified Leakage Risk Score** $\mathcal{L}(f)$ combining correlation, mutual information, and single-feature performance inflation into one flaggable number per feature.
3. **LLM-based semantic leakage analysis**: detecting leakage from feature *meaning* (e.g. `discharge_date`) where statistics alone are blind.
4. **A quantitative benchmark** against three widely used tools on a 29-item checklist and four leakage scenarios.

Validated on **12 datasets** (6 synthetic + 6 real-world) · **325 unit tests** · open source

System Architecture



One YAML config + one CSV in $\rightarrow$ 20 checks in 5 dimensions $\rightarrow$ recommendations, readiness score, and reports out.

Three Interfaces, One Pipeline

- **CLI** — `main.py`: batch runs, CI/CD friendly.
- **Python SDK** — `DatasetChecker`: notebooks and pipelines.
- **Streamlit web app** — seven-tab interactive dashboard for non-programmers.

Plus a `@register_check` **plugin API** for custom, domain-specific checks.

```python
from src.checker import DatasetChecker

checker = DatasetChecker(
    "configs/config.yaml")
report = checker.run(
    "data/titanic.csv",
    target_col="survived")

print(checker.score, checker.grade)
checker.save_report("reports/")
```

The 20-Check Catalog (5 Dimensions)

Dimension	#	Checks
Quality	6	missing values, duplicates, outliers, class imbalance, constant features, low variance
Leakage	5	target leakage, train/test overlap, temporal, ID columns, **unified risk score**
Features	3	correlation structure, MI relevance, distribution shape
Sufficiency	4	sample size, n/p ratio, class support, power
Drift	2	KS test, PSI (covariate + label)

- Each check returns a structured `CheckResult` (pass/fail, severity, affected columns).
- Failed checks map to **20 recommendation handlers** producing *code-level* fixes.
- Optional 21st check: LLM semantic leakage (disabled by default).

Contribution: Unified Leakage Risk Score $\mathcal{L}(f)$

Single statistics miss real leaks; three signals combined are far more robust:

$$\mathcal{L}(f) \;=\; 0.35\,\rho(f) \;+\; 0.35\,\tilde{I}(f,y) \;+\; 0.30\,\pi(f)$$

- $\rho(f)$ — Pearson $|r|$ (numeric) or Cramér's V (categorical) $\in [0,1]$
- $\tilde{I}(f,y)$ — mutual information, normalized by the maximum MI $\in [0,1]$
- $\pi(f)$ — *single-feature performance inflation*: $\dfrac{A_f - A_{\text{base}}}{1 - A_{\text{base}}}$ — how close does this one feature alone get a model to perfect accuracy?

Flagging thresholds

$\mathcal{L}(f) \geq 0.7 \Rightarrow$ **warning** $\qquad$ $\mathcal{L}(f) \geq 0.9 \Rightarrow$ **error**

Are the Weights Arbitrary? — Ablation

Weight-sensitivity analysis on known leaky features (Titanic):

Weight scheme $(\rho/\tilde{I}/\pi)$	$\mathcal{L}(\texttt{name})$	$\mathcal{L}(\texttt{boat})$
Default (0.35/0.35/0.30)	**1.000**	**0.74** ✓
Correlation-heavy (0.50/0.25/0.25)	1.000	0.662 ✗ (below threshold!)
MI- or π-weighted schemes	1.000	flagged ✓

- `boat` (lifeboat number) is a true leak, but its raw correlation is diluted by missing values — an over-weighted correlation term produces a **false negative**.
- Every scheme that gives reasonable weight to MI or performance inflation catches it.
- Empirical evidence that the default weights are *defensible*, not decorative.

Contribution: Semantic Leakage Analysis with an LLM

Why statistics are not enough

- A feature named `discharge_date` in an ICU mortality dataset is leaky *by meaning*: it may show weak correlation, yet it cannot exist at prediction time.
- No statistical test reads feature *names*.

Approach

- Feature names + sample values + dataset description → **GPT-4o-mini** (Azure OpenAI).
- Degrades gracefully without credentials; disabled by default.

Per-feature output

`risk_level`: none / low / medium / high

`leakage_type`: temporal / proxy / post-hoc / indirect

+ natural-language rationale

Raises naming-based leakage detection from **0%** (no statistical signal) to **F1 = 0.963** on a 30-feature labelled benchmark.

Semantic Module: Evaluation and an Honest Caveat

- Manually labelled **30-feature benchmark** across temporal, proxy, post-hoc, and indirect leakage types (`data/semantic_benchmark.json`).
- Results: **Precision = 1.00, Recall = 0.93, F1 = 0.963**.

Transparency

These figures validate the **evaluation harness** using a deterministic mock analyzer — not the live GPT-4o-mini model (Azure credentials were unavailable at evaluation time). A live-model evaluation is the **top-priority item of future work**, and this is stated explicitly in the thesis.

The architecture, prompt design, structured output parsing, and graceful degradation are fully implemented and tested (33 unit tests for Phase 16 modules).

From 20 Checks to One Number: the Readiness Score

$$\text{overall} \;=\; 0.25\, S_{\text{quality}} + \mathbf{0.35}\, S_{\text{leakage}} + 0.25\, S_{\text{features}} + 0.15\, S_{\text{sufficiency}}$$

- Per dimension: start at 100, subtract **15 per error**, **5 per warning**.
- **Leakage carries the largest weight (0.35)**: it is the only failure mode that *silently inflates* results instead of visibly degrading them.
- All weights and penalties are **configurable** in `config.yaml`.

A ≥ 85 | **B** ≥ 70 | **C** ≥ 55 | **D** ≥ 40 | **F** < 40

Experimental Setup: 12 Datasets

6 synthetic (controlled ground truth)

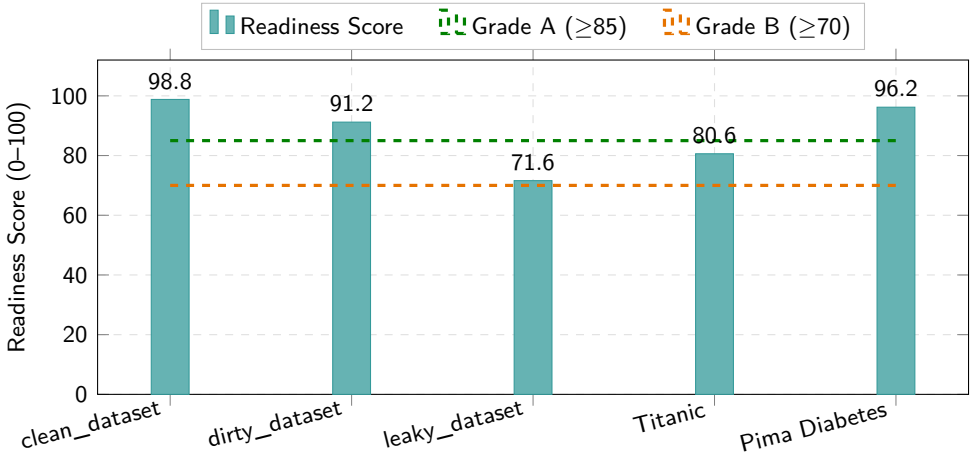
- `clean_dataset` — control
- `dirty_dataset` — quality issues
- `leaky_dataset` — leakage issues
- `proxy_leakage` — graded noisy proxies
- `temporal_leakage_ext` — future feature
- `multitype_leakage` — ICU: proxy + temporal + ID

6 real-world (OpenML / UCI)

- Titanic (1,309 rows)
- Pima Diabetes (768)
- Adult Census (48,842)
- German Credit (1,000)
- Heart Disease (303)
- Wine Quality (1,599)

- Synthetic sets verify the framework **detects what we planted**; real sets verify it **finds surprises in the wild**.
- Full pipeline per dataset: 20 checks + impact analysis (23 results).

Results: Readiness Scores Discriminate as Designed



Only the datasets with **confirmed target leakage** drop to grade B — quality noise alone (dirty) stays A. UCI suite: Adult 92.4, German Credit 97.5, Heart Disease 96.8, Wine Quality 88.6 (all A).

Case Study: Titanic — Finding Real Leaks in a Classic Dataset

- `name`: Cramér's V $= 0.999$ with `survived` $\rightarrow$ flagged as **target leakage (error)**. Unique names act as row identifiers.
- `boat` (lifeboat number): $\mathcal{L} = 0.74 \rightarrow$ **warning**. Assigned *after* the outcome — knowing the lifeboat literally encodes survival.
- Neither is caught by any of the three baseline tools.

Why this matters

Titanic is used in thousands of tutorials and courses. Models "achieving" high accuracy with `boat` included are **textbook post-hoc leakage** — and it takes an automated tool 30 seconds to prove it.

Readiness: **80.6 / B** (14/22 checks passed)

Additional case studies (thesis §5.5): Adult Census (missing values, skew), German Credit (near-zero-MI features), Wine Quality (15% duplicates $\rightarrow$ error).

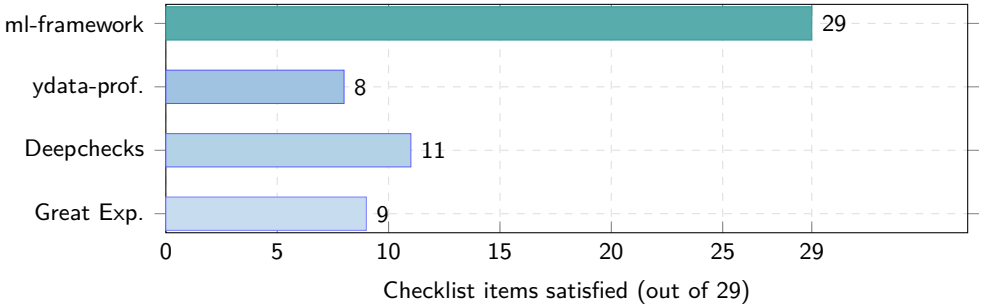
Quantifying the Damage: Impact Analysis

The pipeline retrains the same model *with* and *without* the flagged features (cross-validated):

Dataset (LR)	Baseline acc.	Cleaned acc.	Δ
`leaky_dataset`	1.000	0.952	−**0.048**

- A **positive** baseline−cleaned gap is direct, quantified evidence of leakage — not just a statistical suspicion.
- This closes the loop on the motivating example: the framework *finds* the leak, *explains* it, and *measures* how much of the reported performance was fake.
- Recommendations then emit the exact `pandas` code to drop or re-derive the offending columns.

Benchmark I: Feature Coverage vs. Three Established Tools



- The 29-item checklist is a **cross-tool comparison instrument** (more granular than the 20-check pipeline; the two numbers measure different things).
- **Fairness note**: the baselines target profiling/validation, not leakage — the comparison measures *scope*, and the checklist was published with the benchmark code for scrutiny.

Benchmark II: Leakage Detection — the Decisive Result

Four constructed leakage scenarios, each tool run with default configuration:

Scenario	ml-framework	ydata	Deepchecks	Great Exp.
Perfect proxy (r=1.0)	✓	✗	✗	✗
Noisy proxy (r≈0.97)	✓	✗	✗	✗
ID column (100% cardinality)	✓	✗	✗	✓
Indirect computed feature	✓	✗	✗	✗
Detection rate	**4/4**	0/4	0/4	1/4

- Great Expectations' single detection required a **manually authored rule**; ml-framework's four are fully automatic.
- This is precisely the gap the thesis set out to fill.

Live Demo: Streamlit Web Application

What you will see (`titanic.csv`):

1. Upload the CSV, select `survived` as target.
2. Pipeline runs live: 20 checks + impact analysis.
3. Readiness score **80.6 / B** with per-dimension breakdown.
4. `name` and `boat` flagged with explanations and suggested fixes.
5. Downloadable HTML report.

Run it yourself

```
streamlit run app/app.py
```

Seven tabs: overview, quality, leakage, features, sufficiency, impact, readiness summary.

(Backup: pre-recorded capture and generated HTML report, in case of demo issues.)

Conclusions

1. Dataset quality assessment and leakage detection **can be automated end-to-end**: one config + one CSV $\rightarrow$ scored, explained, actionable verdict.
2. The **unified Leakage Risk Score** detects leaks that no single statistic catches, with ablation-backed weights — **4/4** benchmark scenarios vs. 0–1/4 for existing tools.
3. **Semantic analysis with LLMs** extends detection to leakage that is invisible to statistics; the evaluation harness is built and validated, live-model evaluation is queued.
4. The framework found **real, previously unflagged issues** in classic public datasets (Titanic `boat`/`name`, Wine Quality duplicates).

Engineering: 17 phases · 14 modules · **325/325 tests passing** · 3 interfaces · plugin system

Future Work

1. **Live LLM evaluation** of the semantic module against the 30-feature benchmark (top priority — harness is ready, only credentials are missing).
2. **Broader leakage taxonomy**: group leakage, preprocessing leakage (scaler fit on full data), cross-validation contamination.
3. **CI/CD integration**: readiness score as a merge gate for data pipelines (the SDK already makes this a few lines of code).
4. **Learning thresholds from data**: replacing fixed flagging thresholds with calibrated, dataset-size-aware values.

Thank you

Questions?

Code: `github.com/jaimecano12/ml-framework`

Author: Jaime Cano Moraño

ETSIT-UPM · Illinois Institute of Technology